Services and completion contracts

User interfaces, execution, I/O, file operations, communication, and error detection support applications. Allocation, accounting, protection, and security support coexistence. A service is not merely a syscall name. Kernels, libraries, daemons, and tools can cooperate to provide one interface. Actual privileges and object access determine protection boundaries.

Copy service	Provider in this model	Observable completion
Interpret source/destination	Shell and copy tool	Two pathnames, not kernel interpretation of shell syntax
Find file and check rights	VFS/filesystem	Successful open descriptor or explicit error
Transfer	Library API→kernel I/O	Actual completed bytes, possibly below request
Allocate and protect	Kernel/filesystem	Valid descriptors, cache/storage capacity, rights
Store result	Filesystem/device	File synchronization distinct from directory-entry persistence

Services and completion contracts (continued)

User and system programs normally obey the same protection rules. The name system tool grants no kernel privilege. A privileged user-space daemon still runs in user mode; credentials and CPU mode are separate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A 6000-byte copy with short writes

Source is a fixed6000 B, destination a new empty regular file, buffer 4096 B, blocking I/O, no competing writer.

Event	Returned bytes	Source offset	Destination offset/length	Buffer pending
Open source/create destination		0	0	0
read 4096	4096	4096	0	4096
write 4096	1000	4096	1000	3096
write buffer[1000:]	3096	4096	4096	0
read 4096	1904	6000	4096	1904
write 1904	1024	6000	5120	880
write buffer[1024:]	880	6000	6000	0
read 4096	0, EOF	6000	6000	0

A 6000-byte copy with short writes (continued)

Excluding directory synchronization, the path has2 opens,3 reads,4 writes,1 fsync,2 closes, totaling12 API operations. The essential rule is progress from actual returned bytes. One write after each read would lose3976 B.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Copy-loop invariant and failure boundary

Let R be bytes read, W bytes written, and [j, n) the pending buffer interval. Maintain $W \le R$ and buffer[j: n] equal to the next contiguous source fragment not yet written. A positive return $k \le n-j$ increments both W and j by k without changing sequence. Only j=n permits overwriting the buffer with another read. EOF with no pending bytes proves W=R=6000.

Handle zero-progress EINTR according to the interface and retry the same remaining range. A positive count advances progress first; rewriting the whole chunk duplicates data. Unexpected write=0 must report no progress or otherwise avoid an infinite loop. ENOSPC/EIO leaves partial output and cannot return success.

A successful write is not crash durability. Under a compliant local filesystem/device flush contract, successful destination fsync confirms its data and required metadata. A new name also requires synchronization of its containing directory. This copy still exposes a partial destination on failure. Atomic replacement needs a same-directory temporary file, data synchronization, rename, and directory synchronization protocol; close alone does not supply it.

Shell, GUI, and kernel

A command interpreter translates text/scripts into processes and I/O connections; a GUI translates input events into application actions. Both can use the same kernel services. A child-only `cd` cannot change the parent shell's cwd, which explains its usual builtin implementation. External `cat` can execute in a child. A builtin is not necessarily kernel code. Privileged frontends also expose attack surfaces such as command injection and accidental overwriting.

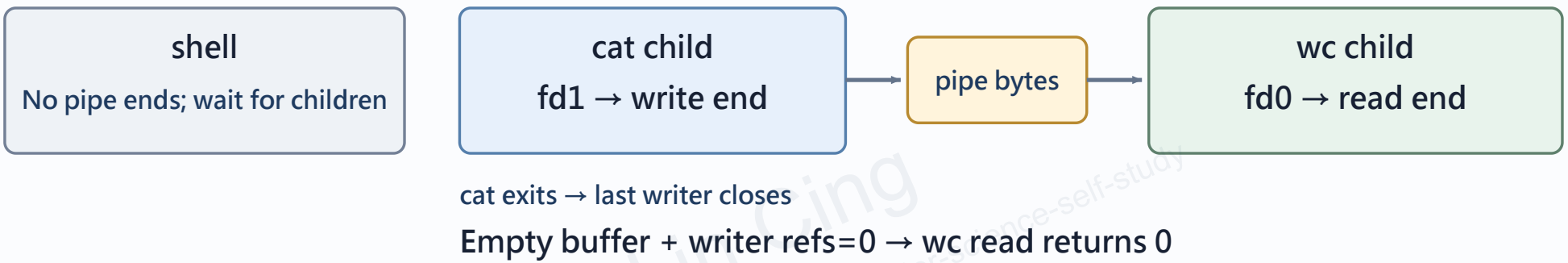
For POSIX-style `cat a | wc -c`, assume no alias, job control, or extra subshell. The shell creates a pipe, forks two children, connects cat's stdout to its write end and wc's stdin to its read end, then waits. Two execs replace child images without creating two more processes. The parent retains its image.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Shell, GUI, and kernel (continued)

cat a | wc -c: pipe references after exec

fd3=reader, fd4=writer. Close original ends after dup2; parent closes both.



Gray: parent. Blue: writer. Green: reader. Arrows: data. EOF is not an extra transmitted byte.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Endpoint references determine EOF

pipe returns fd3 read and fd4 write. Initially all three processes inherit both. Cat uses dup2(4,1), wc uses dup2(3,0), then closes redundant descriptors.

Process	Remaining pipe reference	Close	Reason
Shell	None	3,4	Do not keep a writer alive
cat	fd1→write end	3 and original4	Do not hide loss of readers with own read reference
wc	fd0→read end	Original3 and4	Do not retain a writer that prevents EOF

Cat writes6000 B and exits. Wc can read arbitrary chunks and accumulate6000. Only an empty pipe with every write-end reference closed makes the next read return 0. If the shell retains fd4, wc may block after cat exits while the shell waits for wc. Process exit alone does not establish the reference condition.

Output, status, and permissions

If cat cannot open a and returns ENOENT, it can exit nonzero without writing bytes. Wc can correctly count the empty stream, output0, and exit0. Collect both wait statuses. Pipeline-wide failure follows the shell's specified rule, independently of data correctness.

A GUI cannot claim copy success merely because a progress bar ends. It must await the I/O/error and synchronization contract it promises. CLI versus GUI does not change access checks, capacity, or crash consistency. `cat 'a b'` supplies one pathname; `cat a b` supplies two. The shell parses quotes and pipes. The kernel receives an exec argument array and does not reapply shell syntax.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

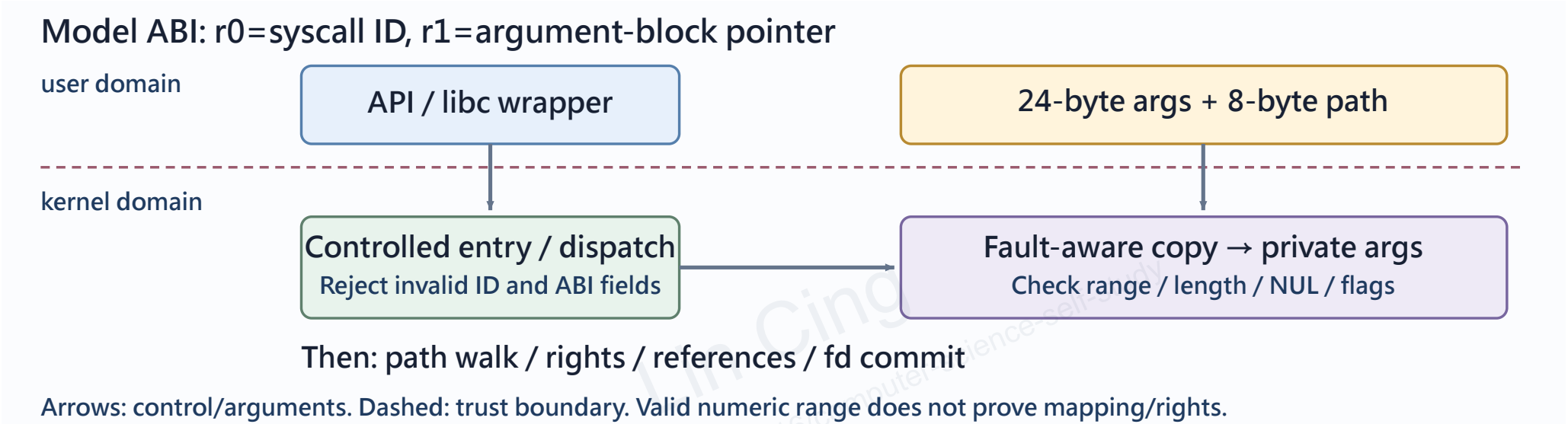
API, ABI, and implementation

An API defines function names, types, and observable behavior. An ABI defines machine calling conventions, syscall numbers, registers/stack layout, and returns. Kernel implementation operates on protected objects. Source compatibility does not make a binary portable across ISAs/ABIs.

`strlen` can run entirely in user space. Buffered `printf` may retain output and combine several calls into one write. A copy API can combine many opens/reads/writes. API-call counts therefore differ from trap counts. If libc converts kernel errors into−1/errno, distinguish the two return formats.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

API, ABI, and implementation (continued)



The teaching ABI puts syscall ID in r0 and a pointer to a24 B argument block in r1. Its three64-bit fields are pathname pointer, flags, and mode. This is a specified model, not a claim that Linux open always uses such a block; real platforms may use registers and vary by version.

Copy arguments before validating their meaning

`/data/x` plus NUL occupies8 B. A fault-aware user copy obtains the24 B fixed block, then bounded string copying obtains at most PATH_LIMIT bytes,8 here. Total copy is32 B. Parse and use the same kernel-owned copy so later user modifications cannot substitute another pathname.

Step	Check	Failure
Decode ID/ABI	Known ID, valid fields/flags	Reject, never jump to arbitrary handler
Copy block	User range and actual fault-safe access	EFAULT or model equivalent, no partial arguments used
Copy pathname	Accessible, NUL-terminated, bounded	EFAULT/ENAMETOOLONG, no published fd
Path walk	Directory search and final-object rights	EACCES/ENOENT or other error
Allocate/publish fd	Stable reference, rollback incomplete allocation	Valid fd or no leaked reference

Copy arguments before validating their meaning (continued)

A readable-looking VMA is insufficient: another thread can unmap it and the actual access can fault. Use fault-aware copying, check that the copy completed, and validate and consume the same kernel-owned bytes. **Bytes stable after copying are not an atomic snapshot of the source during copying.**

For example, user fields begin at (length=4, capacity=4). The kernel reads length=4; user code writes length=8 and then capacity=8; the kernel reads capacity=8. The private result is (4,8), although the source visited only (4,4), (8,4), and (8,8). Treat the result as arbitrary untrusted input and revalidate cross-field relations, lengths, and authority. One copy does not guarantee coherent source fields. An API requiring a coherent snapshot needs an additional enforceable synchronization or version protocol; an adversarial caller cannot be trusted to honor a user-space lock. Filesystem namespace stability during path walk separately requires kernel object references and synchronization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prove arithmetic before accessing memory

For32-bit unsigned words, nonempty ranges in user addresses[0, U), U=0xC0000000, use `p < U && n <= U-p`. Do not first calculate wrapping p+n. Arithmetic success still needs mapping, permission, and fault-safe copy checks.

p	n	Mathematical exclusive end	Result
0xBFFFFF00	0x100	0xC0000000	Valid, final byte0xBFFFFFFF
0xBFFFFF00	0x101	0xC0000001	Crosses boundary
0xFFFFFFF0	0x40	0x100000030	Invalid,32-bit addition disguises it as0x30

For count elements of4 B, first require count≤SIZE_MAX/4. count 0x40000001 has product0x100000004, truncated to4 in32 bits. Allocating4 B and iterating the original count overruns memory. Zero-length pointer rules belong to each API; the present condition explicitly concerns nonempty ranges.

Syscall categories connect objects to state

Category	Requests	Protected state	Success does not imply
Process control	fork, exec, wait	Task, image, exit status	exec creates another process
Files	open/read/write/lseek/close	fd, OFD, inode	Write durability
Devices	read/write/control	Driver queue, device state	Queueing is completion
Information	Attributes, time	Credentials, metadata	Object ID grants rights
Communication	Pipe, messages, shared memory	Endpoint, message, mapping	Stream read is one record
Protection	Permissions, credentials	Policy, principal	Authentication is every authorization

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Syscall categories connect objects to state (continued)

Categories overlap. Socket reads involve I/O, communication, and protection. An fd need not name a regular file. Files, pipes, and terminals share read-like interfaces with different blocking, EOF, and seek rules. Parameters may use registers, blocks, or a stack. Persistent channels and bounded message units have different correctness obligations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Returns are state-machine outputs

For a regular-file read without concurrent modification, count 4096 from offset0 may return 3000 and advance to3000, then1096 and advance to4096, then0 at that moment's EOF. A short read is not permanent EOF; on a pipe it may merely reflect available bytes.

Model kill(pid100, sig9) as two32-bit arguments,8 B of representation, not necessarily8 B copied. The kernel resolves the PID namespace and validates authority/signal. Missing target gives ESRCH; insufficient authority EPERM. Success means delivery/queueing under signal rules, not observed complete termination and reclamation. Wait supplies lifecycle synchronization. Multiple successful syscalls are not automatically one atomic transaction. Identify the commit point; protection, blocking, visibility, durability, and termination remain separate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Source to running process

For hello.c with main and libc output, the path is preprocess→compile→assemble→link→image. Dynamic execution additionally needs a compatible interpreter/loader and libraries. Compiler/linker are user programs using kernel file, memory, and process services; generating machine code does not give kernel mode.

Stage	Input/output	Independent failure
Preprocess	Source/headers→tokens	Missing header, wrong macro branch
Compile/assemble	Tokens→assembly→object	Syntax/type error, target ISA mismatch
Link	Objects/libraries→image	Unresolved symbol, ABI mismatch
Exec/load	Image/interpreter→process image	Permission, format, missing loader
Run	Entry→APIs/syscalls	Fault, I/O error

File tools, status tools, programming support, loading, communication, and background services have different purposes. Daemon lifetime changes neither CPU-mode enforcement nor kernel-interface rules.

Build errors must propagate

For parser, compiler, linker, success is0 and failure nonzero. `parser && compiler && linker` runs a stage only after success. Semicolons continue regardless, and the last status may reflect only the linker. Prevent stale outputs through explicit versions and dependencies, not mere existence.

Case	Parser	Compiler	Linker	Conclusion
All succeed	0	0	0	Image exists, execution still unproved
Parser fails with &&	2	Not run	Not run	Build failed, do not publish stale image
Compiler fails with ;	0	1	0 using stale object	Last0 hides earlier failure

Source API compatibility, binary ABI compatibility, and complete runtime dependencies are separate. Link success or a matching digest alone proves neither a bootable target nor error-free execution.

Mechanism and policy

Timer interrupts, context save/restore, and controlled ready-queue operations are mechanisms. Quantum and selection order are policy. Timer resolution, nonpreemptible code, and switch cost constrain achievable policy latency.

One CPU has A/B/C bursts5/3/1 ms, all ready at0, initial RR order A, B, C, no I/O, zero switch cost.

q	CPU timeline	Completion A, B, C	First response A, B, C
1	A0–1, B1–2, C2–3, A3–4, B4–5, A5–6, B6–7, A7–8, A8–9	9,7,3	0,1,2
4	A0–4, B4–7, C7–8, A8–9	9,7,8	0,4,7

Both do9 ms useful work. Mean response is1 versus11/3 ms. The q 1 trace has7 actual task switches; q 4 has3. Final A→A is no task switch. Nonzero switch cost changes wall times and requires recalculation.

Layered proof requires acyclic dependencies

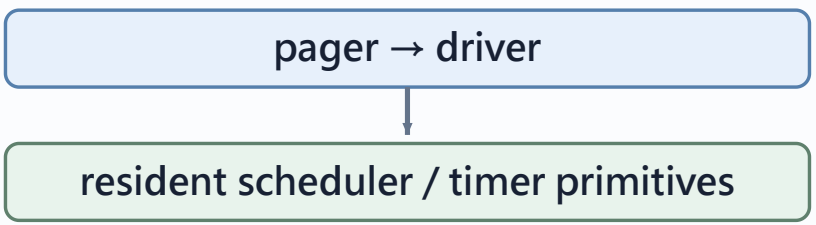
Strict layers use only lower-layer contracts. If layer0 satisfies C0 and each layer i establishes Ci assuming C0…C(i−1), induction establishes upper properties. Tests do not make lower layers infallible; false contracts, premises, or coverage defeat the inference.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Layered proof requires acyclic dependencies (continued)

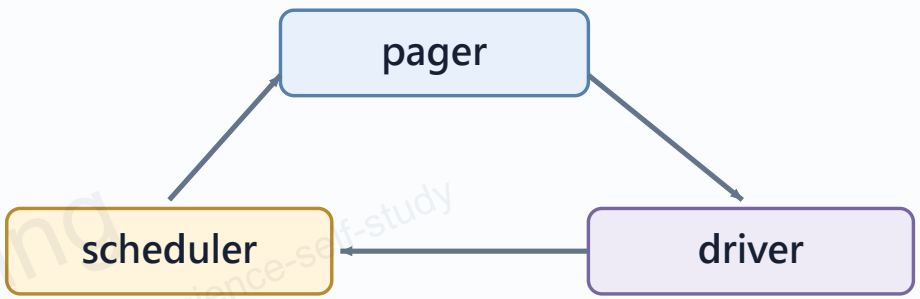
Strict layers require downward dependencies; a cycle has no descending layer order

Valid: pager above resident progress resources



Callee contract holds before caller relies on it.

Invalid: scheduler metadata can still page fault



Arrow: caller depends on callee. pager→driver→scheduler→pager contradicts every layer order.

Pager→disk driver→scheduler→pager cannot have strictly descending layer numbers. Keep necessary scheduler metadata and fault/I/O-progress resources resident, and separate lower scheduling primitives that do not depend on paging. Hiding an arrow does not remove the cycle.

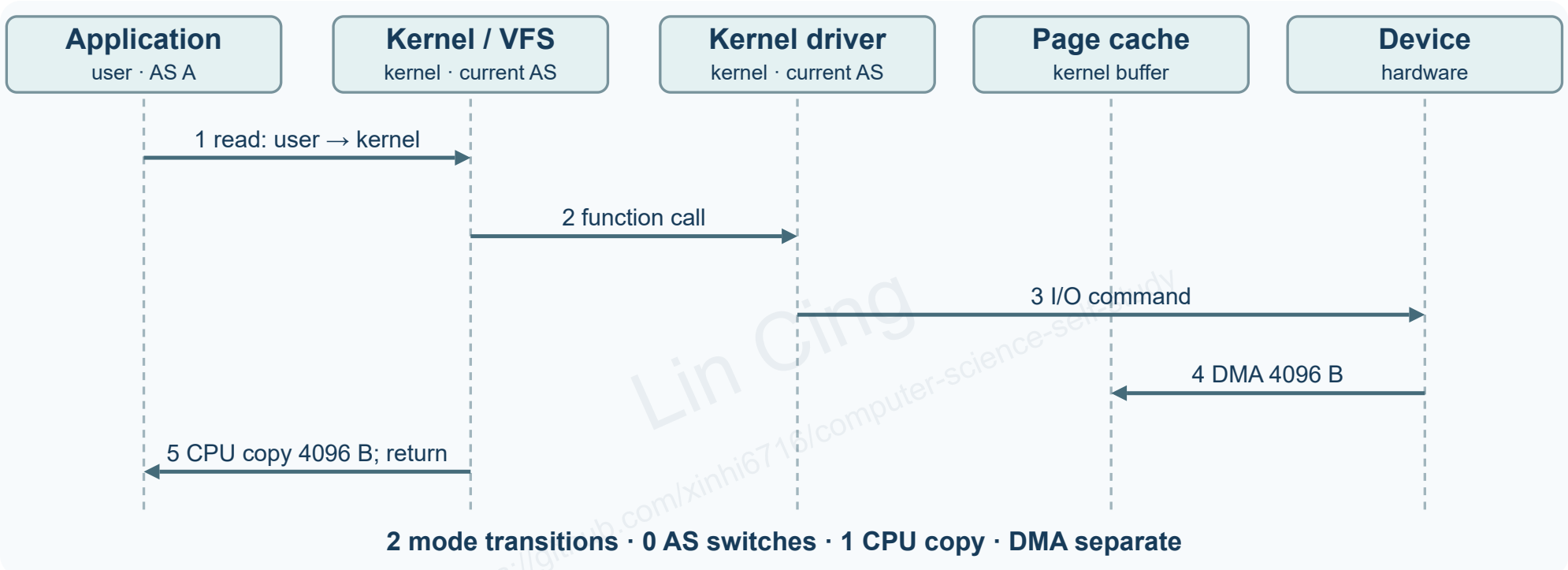
Implementation language and measurable goals

Higher-level languages can improve readability, portability, and optimization. They do not automatically provide memory safety, atomicity, or hardware ordering; C pointer errors can corrupt a kernel. Assembly remains at machine-specific entry, context, and privileged-instruction boundaries. One library API does not erase ABI differences.

With queue capacity1024, identical pregenerated operations, and identical machine resources, candidate A loses no elements. Candidate B loses an enqueued element in one recorded interleaving. B violates correctness even if faster. This is logical exclusion, independent of a throughput threshold.

CPU time per request, end-to-end latency, throughput, and p99 have different denominators. Define workload and quantile rules. Zero-cost schedule tables describe only that model. Real comparison needs switching, cache, and wait measurements, without inventing throughput or p99.

Monolithic read: sequence and boundaries



A fixed 4096 B cold read; follow events downward. A cache hit omits device work but retains the cache-to-user copy.

A fixed cold read in two structures

Use read(fd, user_buf,4096) from `/srv/data.bin`, offset0, cold page cache. ABI arguments use registers, with no additional parameter copy. The monolithic VFS, filesystem, and block driver share kernel execution. In the microkernel model, file server and driver have separate user address spaces, kernel mapping is in the current user AS, and each IPC uses sender→kernel→receiver two-copy delivery.

The application uses libc, enters kernel mode, validates fd/rights/buffer/offset, calls VFS→filesystem→driver, transfers4096 B by DMA into page cache, copies4096 B to user, and returns.

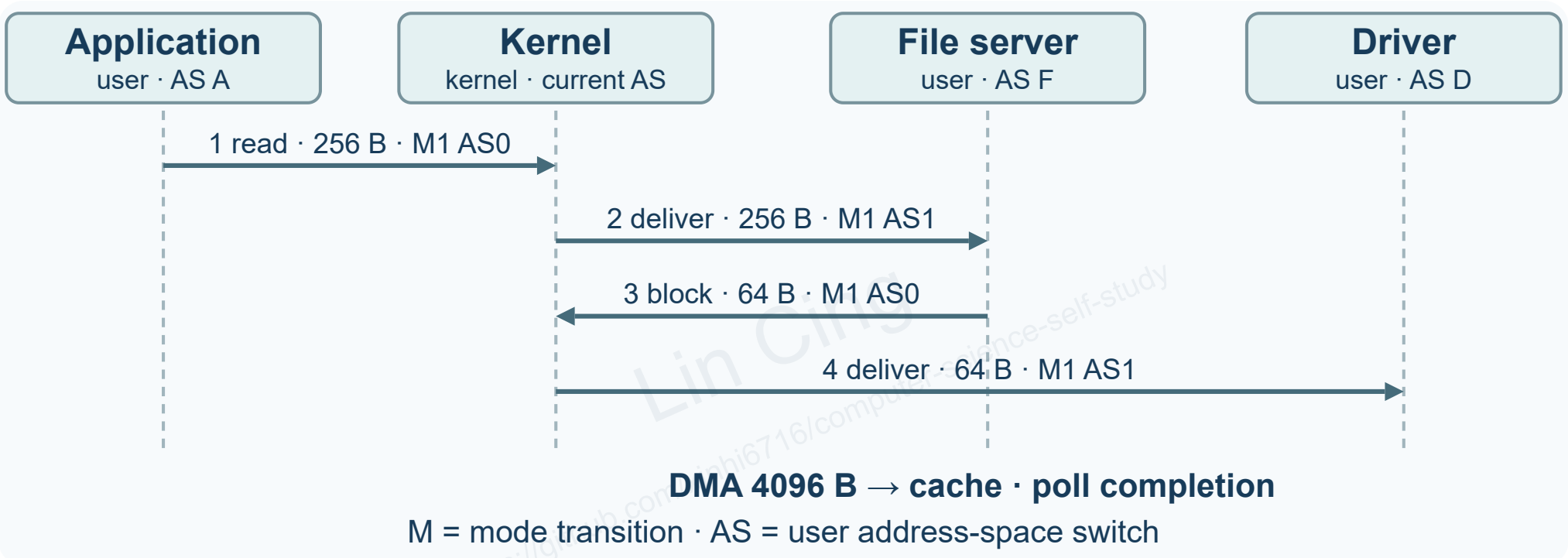
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A fixed cold read in two structures (continued)

Event	Boundary	Cost
libc read	ABI call	0
sys_read	User→kernel, register args	Mode+1
VFS/filesystem/driver	Same-AS function calls	AS+0
Disk→page cache	4096 B DMA	CPU copies+0
Cache→user	4096 B CPU copy	CPU copies+1
Return4096	Kernel→user	Mode+1

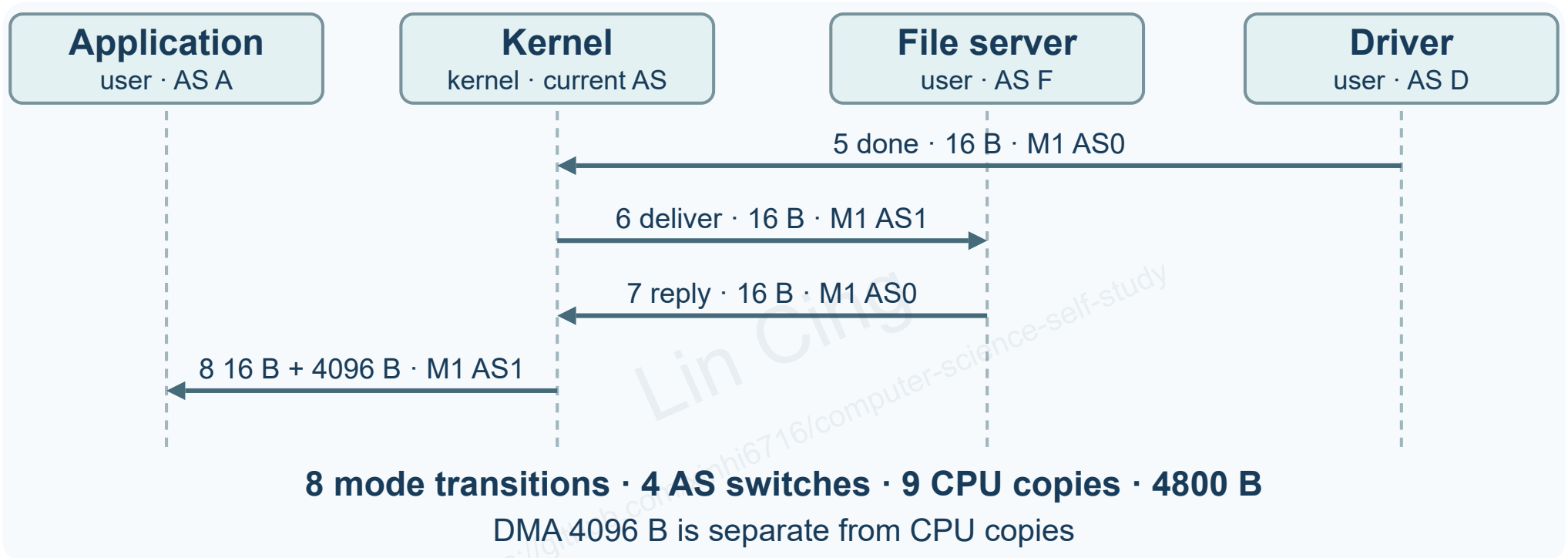
https://github.com/xinhi6716/computer-science-self-study

Microkernel: request and device work



Fixed two-copy IPC; DMA buffer and device authorization are preconfigured.

Microkernel: completion and reply



Fixed two-copy IPC; DMA buffer and device authorization are preconfigured.

Eight microkernel boundary events

App, FS, Driver are three user processes. Entering the kernel retains the current user AS; handing control to another user process changes AS.

Assume the kernel has already allocated and pinned the DMA buffer and authorized device access and mappings for the driver. Setup and teardown are outside the counts; an IOMMU, if used, must enforce the device mapping.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Eight microkernel boundary events (continued)

Step	Boundary	IPC action	Control copy	Mode	AS	CPU copies
1	App→Kernel	SYS_IPC_SEND(read_req), register args	256 sender→kernel	1	0	1
2	Kernel→FS	Deliver request	256 kernel→receiver	1	1	1
3	FS→Kernel	SYS_IPC_SEND(block_req)	64 sender→kernel	1	0	1
4	Kernel→Driver	Deliver block request	64 kernel→receiver	1	1	1
5	Driver→Kernel	Completion send	16 sender→kernel	1	0	1
6	Kernel→FS	Deliver completion	16 kernel→receiver	1	1	1
7	FS→Kernel	Reply send	16 sender→kernel	1	0	1
8	Kernel→App	Deliver reply and copy cache	16 kernel→receiver plus4096data	1	1	2

Eight microkernel boundary events (continued)

Total8 modes,4 user-AS switches,9 CPU copies. Four messages256/64/16/16 B take8 copies and704 B. Add one4096 B data copy for4800 B. One 4096 B DMA remains separate. AS handoffs are App→FS, FS→Driver, Driver→FS, FS→App.

A function call is not automatically zero-copy, nor does all microkernel IPC require these eight transitions. Merging servers, private kernel AS, shared memory, or one-copy delivery requires a new ABI/deployment/count table.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DMA boundary and model sensitivity

Baseline counts cover only the listed request-chain handoffs. DMA completion uses polling, no other runnable work exists, and no timer preemption or completion interrupt occurs. Interrupt entry/exit, wakeup, and scheduling need separate rows if introduced.

Change	Mode	AS	CPU copies	Bytes copied
Monolithic, current-AS kernel	2	0	1	4096
Microkernel,256/64/16/16	8	4	9	4800
Microkernel request shrinks to128	8	4	9	4544
FS and driver in one process	4	2	5	4640
Monolithic, private kernel AS	2	2	1	4096
Microkernel, private kernel AS	8	8	9	4800

DMA boundary and model sensitivity (continued)

The128 B request gives2×128+2×64+4×16=448 control bytes, plus4096=4544. Merged FS/driver retains256 B request and16 B reply,544 control bytes plus4096=4640. Private-kernel-AS changes AS at every mode transition in both architectures. These are model sensitivity results, not universal syscall costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

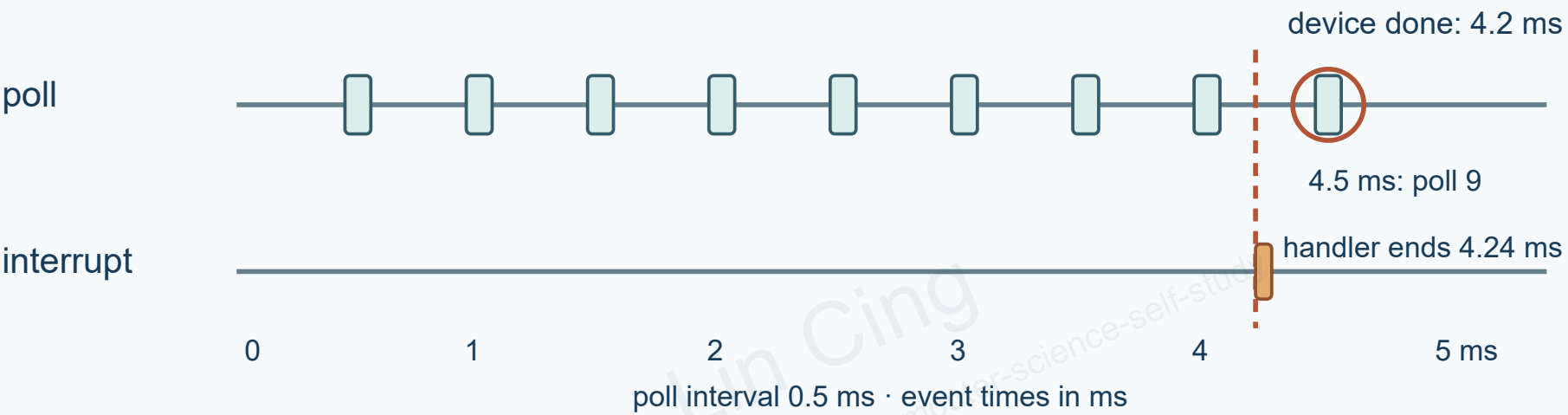
How is I/O completion discovered?

Polling has the CPU read device status; an **interrupt** notifies the CPU to run a handler; **DMA** moves data between a device and memory. DMA completion can be discovered by polling or by interrupt: DMA is not a third mutually exclusive notification method.

Method	CPU while waiting	Detection and cost
Busy polling	Repeated checks occupy the core	May detect quickly but consumes CPU
Periodic polling	Checks on a timer; other work can run between checks	Detection can wait until the next check; each check and timer costs work
Interrupt	Device notifies CPU on completion	No continuous checks, but handling and synchronization still cost work

Real devices may batch notifications, combine methods, or poll under high load. Workload, hardware, and scheduling determine which approach performs better.

A fixed polling and interrupt timeline



Suppose a device completes at **4.2 ms**. Periodic polls start at 0.5, 1.0, …, 4.5 ms and each consumes **0.01 ms** of CPU. Assume reliable status at the check instant and no scheduling delay. The ninth poll detects completion at 4.5 ms: detection delay is 0.3 ms and nine checks use 0.09 ms of CPU.

A fixed polling and interrupt timeline (continued)

In a separate model, interrupt delivery is immediate and the handler takes 0.04 ms, ending at 4.24 ms. Submission, scheduling, and return-to-application costs are excluded. Busy waiting from 0 to 4.2 ms instead consumes 4.2 core-ms; the core is not idle.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A wakeup does not immediately run the thread

After completion, the poller or interrupt handler must establish required data visibility and update completion state. A waiting thread may transition **waiting → ready**. It becomes **ready → running** only after selection by the scheduler and restoration of its context.

In the preceding interrupt model the handler ends at 4.24 ms and makes P ready. If another thread yields and P runs at 4.34 ms, P has waited 0.10 ms in the ready queue. Handler completion at 4.24 ms and P's execution at 4.34 ms are different milestones; return to the application and data copying can require more work.

Safe DMA-buffer reuse also needs device completion, required synchronization, and ownership transfer. Early reuse can let the device and a new owner modify the same memory concurrently.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Four independent structural dimensions

Structure	Calls/dependencies	Privilege and AS	Extension	Failure contract
Monolithic	Mostly kernel function calls	Shared kernel privilege	May load modules	Bad pointer can corrupt kernel
Strict layers	Only lower interfaces	Layer need not be protection domain	Static or dynamic	Contracts support induction, not automatic isolation
Microkernel	Small kernel plus server IPC	Isolated user domains	Replace servers separately	Parsers, authority, recovery still need correctness
Modular kernel	Defined module interfaces	May share kernel privilege	Runtime load/unload	Maintainability is not fault isolation
Hybrid	Deployment varies by subsystem	Label each actual domain	Mixed module/server	Name determines neither cost nor security

Four independent structural dimensions (continued)

Layers describe dependencies, modules organization/linking, and protection domains access control. Separate files/modules create no automatic user/kernel boundary. Stacked boxes do not prove acyclic dependencies.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Isolation and recovery are separate

A file server crashes after receiving append R but before replying. It may have crashed before commit or committed and lost the reply. Retrying repairs the first history but duplicates the second. Memory isolation alone cannot distinguish them.

Recoverable request IDs and dedup/result records can return the same result on retry. Records must persist atomically with the append or crashes can still break consistency. Limit device, memory, and handle authority. A server with unrestricted disk writes can corrupt disk data even when unable to overwrite kernel memory.

Module unloading separately must stop new entry and wait for references, DMA, callbacks. Removing a symbol-table entry does not stop old function pointers. This differs from restarting a user-space server.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Historical hybrid systems

The textbook's Mac OS X has Aqua/Cocoa above a kernel environment containing Mach, BSD, I/O Kit/extensions. Mach's presence does not imply all BSD services are separate user processes. The historical iOS stack has Cocoa Touch, Media Services, Core Services, Core OS.

The historical Android stack is applications→framework→libraries/Dalvik→Linux kernel. Dalvik is the textbook runtime, not a timeless claim about all Android. Linux supplies process, memory, device services without implementing every Java API's high-level meaning. Compare the contract, domain, boundary of each component, not stack names or assumptions about later versions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Evidence at failure time

A user core preserves some process state; a kernel crash dump preserves kernel failure state. Logs contain selected events; traces expose recorded calls, scheduling, I/O ordering. None alone reconstructs every past event.

Evidence	Can answer	Cannot alone establish
PC, registers, stack	Failing instruction/call chain	First corruption happened there
Object/heap snapshot	Values at crash	Who changed them and when
Timestamped trace	Recorded order and timing	Unprobed events never occurred
CPU sampling	CPU-active stacks	Every long I/O wait was sampled

A broken filesystem may also fail when asked to store its kernel's dump. Reserve a reliable dump path/space and retrieval procedure. Collection failure does not establish absence of failure.

Decompose a 10 ms request

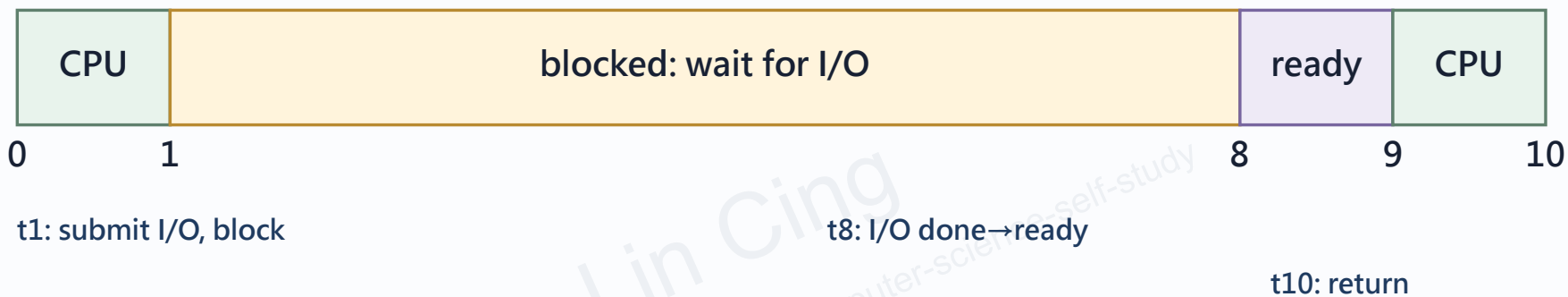
Assume comparable monotonic time and no lost events. Handler starts0, blocks for I/O at1, becomes ready at8, runs at9, returns 10. CPU[0,1)+[9,10)=2 ms; blocked[1,8)=7 ms; ready wait[8,9)=1 ms; total10 ms.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Decompose a 10 ms request (continued)

One 10 ms request: CPU 2 + blocked 7 + ready wait 1

Scale: 1 ms=100px. Intervals include left, exclude right endpoints.



Green: CPU. Yellow: blocked. Purple: ready, not selected. Completion need not return to user immediately.

Seven blocked milliseconds are not disk CPU time. Device completion at8 is not return at8. Probes can change scheduling, so compare observable behavior with uninstrumented execution. For1,1,1,1,1,1,1,100 ms, mean107/8=13.375 ms. Nearest-rank p99 uses ceil(.99×8)=8 and yields100 ms. This small sample's order statistic is not a stable population-tail estimate.

Instrumentation can conceal a race

From x=0, T1: r0→T2: r0→T2: w1→T1: w1 produces1; serial increments produce2. printf can serialize the execution without repairing synchronization. Correctness requires indivisibility for all allowed interleavings, through a mutex covering the whole RMW or a suitable atomic RMW. Locking writes alone leaves both initial reads0 possible.

Dynamic probes must retain event IDs, time domain, drop counts, and necessary object identities. State assumptions about ordering per-CPU timestamps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

System generation and dependency closure

SYSGEN can select source at compile time, precompiled components at link time, or runtime functions through tables/discovery. These trade image size, portability, hardware-change cost, and early availability. Recompilation alone does not classify the design.

Configuration	Requirement	Failure if missing
CPU/ABI	Compatible kernel/user ISA/ABI	Unsupported instruction or exec format
RAM/reservations	Avoid firmware/device-reserved ranges	Corruption or invalid mapping
Boot storage	Controller driver reads root device	Root device unavailable
Filesystem	Root-format driver available early	Device exists but mount fails
Root/init	Correct root, executable init, loader	Mount works but userspace cannot start

Discovery still needs a usable driver. A module's existence establishes neither version/ABI compatibility nor its availability before use.

Root drivers cannot exist only on inaccessible root

Root resides on storage S with filesystem F. Other dependencies are built in. B means built-in, I an accessible compatible initramfs module, R root-only.

S	F	Initramfs	Root mount reachable?
B	B	Neither needed	Yes
I	I	S, F, and all prerequisites	Yes, load first
R	B	S absent	No, need root to obtain root driver
B	R	F absent	No, device exists but format unavailable
I	I	Only F	No, closure incomplete

Root drivers cannot exist only on inaccessible root (continued)

Obtain root-mount drivers before root becomes readable



Invalid: S exists only on the unmounted root filesystem



Top: feasible order. Bottom: circular prerequisites. Signatures/file existence cannot remove the cycle.

Start from built-ins and readable initramfs components. Repeatedly add components whose prerequisites are all available. If mount-root never enters the set, a dependency is absent or cyclic. Closure only excludes this failure class; hardware, driver logic, and root contents still require correctness.

Reset to init

Assume firmware finds a loader, which loads kernel/initramfs and provides memory map/command line. These are roles; BIOS single-sector and UEFI implementations are not one identical flow.

Stage	Input	Next-stage premise
Reset/firmware	Reset entry, hardware	Usable CPU/RAM, executable loader
Loader	Configuration, images	Correct placement and parameters
Early kernel	Boot/hardware information	Tables, interrupts, scheduler, memory
Root setup	Storage/filesystem drivers	Mounted root, readable user files
Userspace init	Image, interpreter, libraries	Services/login under their policies

Linuxv3.2 start_kernel initializes early infrastructure; rest_init creates the task running kernel_init. It does not directly launch an arbitrary shell. Kernel_init completes initcalls/root preparation and tries the appropriate userspace init. Executable initramfs `/init` can run first; otherwise root namespace preparation is required. Failure points leave different diagnostics.

Locate the failed handoff

Use role gates F→L→K→R→I for firmware, loader, kernel/root access, readable init, and successful exec.

Fault	Last successful stage	First failing action	Incorrect diagnosis
Loader cannot find kernel	L	Load K	Init services failed
Missing S driver	K	Read root device	Only a misspelled root path
Missing F driver	K	Mount existing device	Disk absent
Missing init interpreter	R	Exec init	Kernel never started

This role table does not claim every Linux failure returns one exit code to firmware. Panic, loader error, and userspace nonzero exit are different failure paths.

Verification, measurement, and compatibility

Verified boot has a trusted predecessor authenticate/authorize the next stage and refuse execution on failure. Computing or recording a digest is measurement without automatic enforcement. Policy must name covered images/configuration/initramfs. A kernel filename alone is insufficient.

A validly signed old/revoked version may still be unacceptable, requiring rollback/revocation policy. Signatures do not remove authorized code's vulnerabilities. ABI compatibility and dependency closure support compatibility, not provenance. This abstract trust chain is separate from Linuxv3.2's control flow; it does not claim mainline3.2 implements every policy or that init immediately drops all capabilities. Service authority depends on actual work and security policy.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

One read crosses several contracts

The4096 B cold-read API supplies fd, buffer, count; ABI selects registers; controlled entry establishes kernel context; validation and VFS/filesystem resolve objects; driver/DMA fills cache; the kernel copies4096 B and returns. Function calls, mode transitions, AS handoffs, CPU copies, and DMA are distinct events.

A microkernel can retain the API while adding internal IPC. GUI replaces the frontend without removing read protection. Modules change loading without automatically reducing privilege. Returned bytes mean this read's data is available, not that later writes are complete or durable. Pipe EOF requires empty buffers and no writers; missing server replies do not prove nonexecution.

Follow each object’ s contract and state across handoffs.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapters 1–2. write(2); fsync(2); pipe(7); Linux 3.2 init/main.c.

One read crosses several contracts (continued)

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study